DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Proxy inverso y balanceo

Tema 3 — Publicación y ejecución de aplicaciones web



Objetivo de la sesión

Punto de partida

Nginx sirve archivos y recibe las conexiones públicas.

La aplicación atiende peticiones HTTP en su puerto interno.

Qué incorporamos

Reenvío y cabeceras de la petición.

Reparto entre réplicas y tratamiento de fallos.

Almacenamiento accesible desde cualquier réplica.

Una entrada pública puede atender peticiones mediante varias aplicaciones internas.

Contenido estático y proxy inverso

Nginx: servir contenido o reenviar peticiones

Un mismo punto de entrada, dos recorridos distintos



El navegador accede al mismo origen público; los destinos internos quedan detrás de Nginx.

Ejemplo simplificado:
rutas y puertos ilustrativos.

Proxy directo y proxy inverso

Proxy directo

Se sitúa del lado del cliente.

Representa al cliente frente al destino.

Ejemplo: proxy de salida de una organización.

Proxy inverso

Se sitúa delante de los servidores.

Recibe las peticiones del servicio público.

Ejemplo: Nginx delante de una aplicación web.

En esta sesión, Nginx representa al servicio publicado.

Rutas y responsabilidades

Archivos del frontend

/
/css/...

Nginx obtiene el contenido desde el sistema de archivos.

Peticiones de la aplicación

/api/...

Nginx reenvía al backend.

La aplicación ejecuta su lógica y genera la respuesta.

El navegador utiliza el mismo origen público para acceder a distintos componentes.

Configuración mínima del reenvío

```
location /api/ {
    proxy_pass http://backend:8080;
}
```

location selecciona las rutas que empiezan por /api/.

proxy_pass indica el servidor al que Nginx reenvía la petición.

Ejemplo sencillo con un location de prefijo, sin reescrituras.

La ruta que recibe el backend

proxy_pass: una barra cambia la ruta

Misma petición y mismo location; distinta URI en el destino

El navegador solicita: /api/productos

```
location /api/ { ... }
```

1 · Sin URI en proxy_pass

```
proxy_pass http://backend:8080;
```

Se conserva la ruta de la petición.



2 · Con URI / en proxy_pass

```
proxy_pass http://backend:8080/;
```

El prefijo /api/ se sustituye por /.

La ruta recibida por el backend es /productos.



Elige la forma que coincida con las rutas que espera tu aplicación.

Ejemplo con location de prefijo, sin rewrite ni variables en proxy_pass.

Información original y cabeceras reenviadas

Detrás del proxy: conservar la información original

Ejemplo: Nginx es la primera y única puerta de confianza



Dato original	Cabecera reenviada	Valor del ejemplo
Nombre solicitado	`Host`	`aula.example`
IP observada por Nginx	`X-Forwarded-For`	`203.0.113.25`
Protocolo externo	`X-Forwarded-Proto`	`https`

```
proxy_set_header Host $host;
proxy_set_header X-Forwarded-For $remote_addr;
proxy_set_header X-Forwarded-Proto $scheme;
```

Una cabecera enviada por el cliente no demuestra su IP

En esta arquitectura, Nginx sobrescribe `X-Forwarded-For` con la dirección que observa.

La aplicación debe interpretar estas cabeceras y confiar **solo en los proxies configurados.**

Confianza en el proxy

```
proxy_set_header Host              $host;
proxy_set_header X-Forwarded-For   $remote_addr;
proxy_set_header X-Forwarded-Proto $scheme;
```

Un cliente puede inventar el valor de X-Forwarded-For.

En este ejemplo, Nginx lo sustituye por la dirección observada.

La conexión del backend sigue procediendo de Nginx.

La aplicación debe interpretar las cabeceras y confiar en los proxies configurados.

Un grupo upstream para las réplicas

```
upstream backend_pool {
    zone backend_pool 64k;
    resolver 127.0.0.11 valid=5s ipv6=off;

    server app-1:8080 resolve;
    server app-2:8080 resolve;
    server app-3:8080 resolve;
}
```

Ejemplo en una red de Docker Compose y Nginx 1.27.3 o posterior.

El bloque upstream se declara en el contexto http, junto a los bloques server.

Reparto y observación de las respuestas

Réplicas equivalentes

Cada copia puede atender la misma clase de petición.

El reparto no garantiza una secuencia exacta 1–2–3.

Las peticiones pueden tener costes distintos.

Identificar quién responde

Un endpoint puede devolver el identificador de la réplica.

Ejemplo: /api/instancia

La aplicación debe implementarlo. Nginx no crea ese endpoint.

Round robin con pesos iguales reparte tráfico entre los destinos disponibles.

Nombres estables e IP que pueden cambiar

app-2 conserva su nombre al recrearse, pero puede recibir otra IP.

```
resolver   Consulta el DNS interno de Docker: 127.0.0.11.

resolve    Actualiza las direcciones del grupo según el DNS.

zone       Comparte el estado del grupo entre procesos de Nginx.
```

valid=5s fija la validez de las respuestas DNS almacenadas.

Resolver un nombre permite localizar una réplica. Su disponibilidad exige otra comprobación.

Reenvío al grupo y límites de los intentos

```
location /api/ {
    proxy_pass http://backend_pool;

    proxy_connect_timeout 2s;
    proxy_next_upstream error timeout;
    proxy_next_upstream_tries 3;

    # Aquí se añaden las cabeceras reenviadas.
}
```

Máximo de tres intentos en total, incluido el inicial.

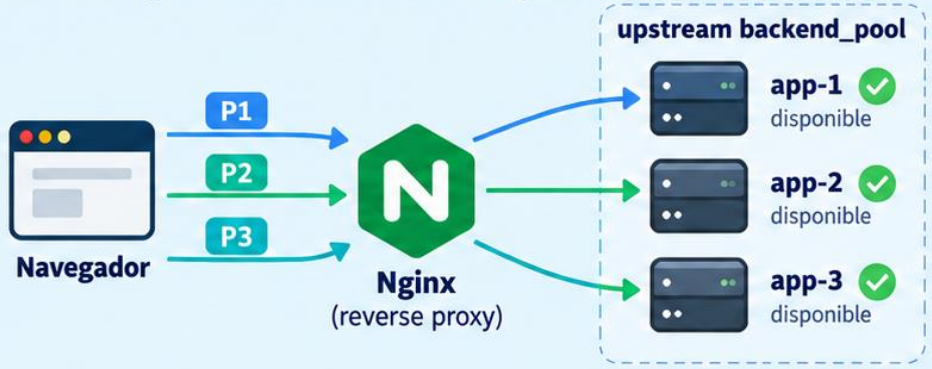
El límite de conexión no limita el tiempo total de respuesta.

Balanceo y detección pasiva de fallos

Balanceo: repartir peticiones y reaccionar a fallos

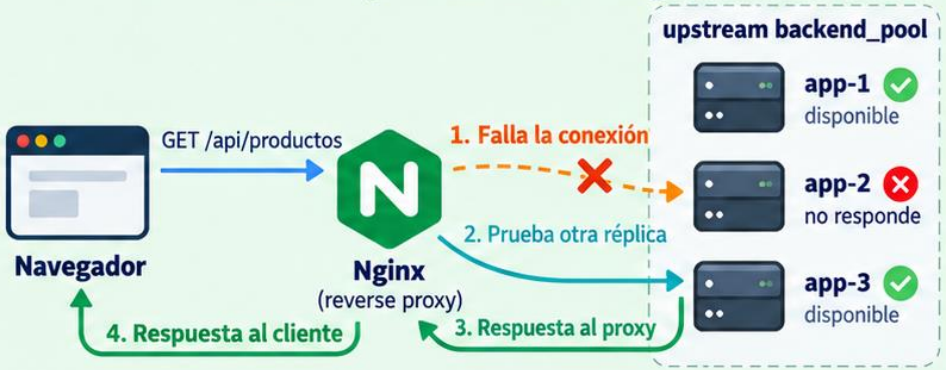
Tres réplicas equivalentes detrás de un único punto de entrada

1 · Reparto entre réplicas



Round robin con pesos iguales: reparto entre destinos disponibles.
Secuencia ilustrativa, no una garantía de orden exacto.

2 · Una réplica no responde



Detección pasiva: el fallo aparece al intentar usar la réplica.

`proxy_connect_timeout 2s` → limita el establecimiento de la conexión.

`proxy_next_upstream error timeout` → permite cambiar de destino ante esos fallos, si se puede reintentar.

`proxy_next_upstream_tries 3` → máximo de 3 intentos en total, incluido el inicial.

⚠ **No toda petición puede repetirse:** un POST ya enviado no se reintenta normalmente; tampoco si ya se envió parte de la respuesta al cliente.

✓ **Nginx reparte tráfico; el healthcheck de Compose no decide el destino HTTP.**

Límites de la recuperación ante fallos

Reintentos condicionados

Solo antes de enviar respuesta al cliente.

Un POST ya enviado normalmente no se repite en otro backend.

error timeout no incluye por sí solo HTTP 500 o 503.

Exclusión temporal

max_fails y fail_timeout controlan los fallos pasivos.

Valores predeterminados: 1 fallo y 10 segundos.

Actualizar DNS no demuestra que la réplica esté preparada.

Varias réplicas reducen algunos fallos, pero no garantizan que toda petición tenga éxito.

Healthcheck y selección del destino HTTP

Docker y Compose

Docker ejecuta una comprobación periódica.

Compose puede usar service_healthy para coordinar el arranque.

El estado no provoca por sí solo un reinicio.

Nginx en este ejemplo

Selecciona el destino de cada petición.

Detecta fallos al tramitar tráfico real.

Puede excluir temporalmente un destino y probar otro.

El estado unhealthy no retira automáticamente una réplica de este Nginx.

Archivos locales y almacenamiento compartido

Varias réplicas: ¿dónde están los archivos?

La misma ruta no significa el mismo almacenamiento

1 · Archivos locales separados



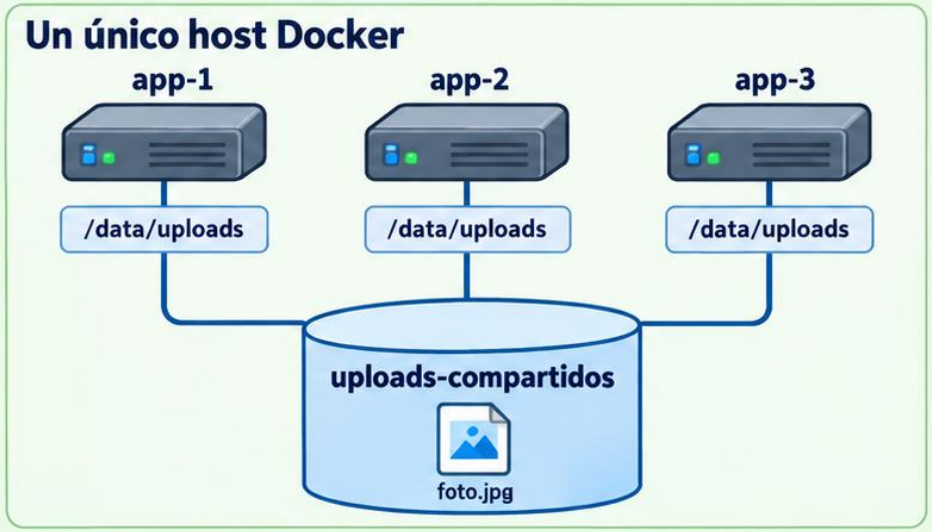
Subida atendida por app-1
Archivo guardado

Lectura atendida por app-2
404: aquí no existe

El balanceador puede enviar la siguiente petición a otra réplica.

Ejemplo: la aplicación devuelve 404 si falta el archivo.

2 · Volumen común en el mismo host



Las tres réplicas acceden al mismo archivo.

En cada servicio de Compose:

```
volumes:
  - uploads-compartidos:/data/uploads
```

El volumen también se declara en la sección superior `volumes`.

Compartir la base de datos no comparte automáticamente los archivos.

En hosts diferentes: almacenamiento de red u objetos.
Un volumen local no se comparte entre máquinas.

También hay que gestionar permisos y escrituras simultáneas.

Montaje del volumen común

```
services:
  app-1:
    volumes:
      - uploads-compartidos:/data/uploads
  app-2:
    volumes:
      - uploads-compartidos:/data/uploads
  app-3:
    volumes:
      - uploads-compartidos:/data/uploads

volumes:
  uploads-compartidos:
```

Cada réplica monta el mismo volumen en la ruta que utiliza la aplicación.

Condiciones del almacenamiento compartido

Dentro del mismo host

El volumen local común permite ver los mismos archivos.

La aplicación necesita permisos adecuados.

Debe gestionar escrituras simultáneas.

Cuando hay varios hosts

Un volumen local no se comparte entre máquinas.

Se necesita almacenamiento de red u objetos.

Compartir almacenamiento no sustituye las copias de seguridad.

El estado que necesita cualquier réplica debe estar accesible desde todas ellas.

Una entrada pública y sus límites

Política común de entrada

Nginx concentra el tráfico público.

Las réplicas y la base de datos se comunican por la red interna.

TLS y registros pueden centralizarse.

Disponibilidad del conjunto

Varias réplicas de app no hacen redundante todo el sistema.

Un único proxy, host o servidor de base de datos puede seguir siendo un punto único de fallo.

Siguiente sesión: HTTPS y control de acceso en Nginx.

Cierre de la sesión

Qué debemos poder explicar y comprobar

Qué ruta recibe el backend y qué cabeceras necesita.

Cómo se reparten las peticiones y se actualizan las IP.

Cuándo puede reintentarse una petición y qué límites tiene.

Por qué un archivo local falta en otra réplica.

Qué resuelve un volumen compartido dentro de un mismo host.

En la actividad aplicaremos el patrón a varias réplicas de una aplicación.